PL/1 wire format

PL/1 is a small binary request/reply protocol carried over a single persistent TCP connection. It is not HTTP and does not reuse an HTTP parser; a peer that keeps the connection open can send many asks and get back many replies in order.

Packet header (12 bytes, big-endian)

Offset	Size	Field	Notes
0	2	Magic	ASCII `PL`
2	1	Version	`1`
3	1	Type	`1` = ask, `2` = reply
4	1	Flags	currently always `0`
5	1	Field count	number of encoded fields that follow
6	2	Reserved	must be `0`
8	4	Payload length	byte length of everything after the header

A receiver that sees a `type` it does not recognize still knows the payload length, so it skips that many bytes and keeps reading the next packet instead of closing the connection. This lets a later protocol revision add new packet types without breaking older peers.

Ask payload

```
method (1 byte)
path length (2 bytes) + path bytes
encoded fields (field_count entries, see below)
body length (4 bytes) + body bytes
```

`method` is `1` for `GET`; no other method is currently defined, so anything else is rejected by the host with status `405`.

Reply payload

```
status (2 bytes)
encoded fields (field_count entries, see below)
body length (4 bytes) + body bytes
```

Field encoding

Ten common field names live in a shared static table so they can be referenced by a single index byte instead of being spelled out on the wire:

```
1  host             6  connection
2  user-agent       7  server
3  accept           8  date
4  content-length   9  cache-control
5  content-type     10 transfer-encoding
```

Each field is encoded as:

- one index byte – `1..10` for a table name, or `0` followed by a one-byte length and the raw name bytes for anything not in the table
- a two-byte value length, then the value bytes

Routes served by `host`

```
GET /add?a=<int>&b=<int>  -> 200 <a+b>
GET /sub?a=<int>&b=<int>  -> 200 <a-b>
GET /mul?a=<int>&b=<int>  -> 200 <a*b>
GET /div?a=<int>&b=<int>  -> 200 <a/b>
```

Any other path is treated as a request for a file under the document root passed to `host` on the command line.

Status codes

- `400` – malformed packet, missing `host` field, bad/missing/duplicate query parameters, division by zero, arithmetic overflow, a non-empty ask body, or a path that resolves outside the document root.
- `404` – an unrecognized calculator route, or a file that does not exist.
- `405` – any method other than `GET`.

See `annotated-capture.md` in this folder for a real byte-by-byte example of one ask/reply pair.